Documento Oficial — Pesquisa Operacional

Projeto: Otimização de Rota de Visitas (TSP com Heurística 2-Opt)
Trilha 5: Caixeiro Viajante (TSP) e Roteamento
Equipe: Guilherme Caleffi, João Franco e Marcio Henrique

1. Cenário e Dataset

Problema (Cenário):
Uma equipe comercial define, para um dia de viagem externa, um conjunto de X clientes que precisam ser visitados. A partida e o retorno ocorrem obrigatoriamente no mesmo ponto de apoio logístico (hotel, aeroporto ou escritório central). Deseja-se determinar a ordem sequencial de visita que minimize a distância total percorrida pelo representante, garantindo que cada cliente selecionado seja visitado exatamente uma vez antes do retorno à base.
Dataset Utilizado:
Para a modelagem e testes, utiliza-se a instância pública clássica **Solomon C102**, amplamente reconhecida na literatura de Pesquisa Operacional para problemas de roteamento.

- A base original contém 100 clientes e 1 depósito.
- O registro com identificador `cust_no = 0` representa o depósito fixo (Hotel/Base).
- Os demais registros formam o *pool* de clientes elegíveis, possuindo coordenadas cartesianas (x, y) para cálculo de distâncias.
- **Recorte do Projeto:** Para garantir clareza didática e visualização gráfica limpa (conforme exigido pela disciplina), o algoritmo permite selecionar dinamicamente um subconjunto de X clientes (ex: X = 15) a partir desse *pool* para calcular e exibir a rota otimizada.

2. Mecânica do Algoritmo

Por que se aplica:

O cenário descrito é a definição acadêmica clássica do Problema do Caixeiro Viajante (*Traveling Salesman Problem* - TSP) com rota fechada e depósito fixo. Como o espaço de busca de soluções cresce fatorialmente (X!) com o aumento do número de clientes, métodos de resolução exata tornam-se computacionalmente inviáveis. Portanto, adota-se a abordagem heurística construtiva seguida de busca local, conforme previsto na Trilha 5 da disciplina.

Como o algoritmo calcula (Passo a Passo):

1. **Heurística Construtiva (Vizinho Mais Próximo):**
    - O algoritmo inicia no depósito (0).
    - Dentre todos os clientes ainda não visitados, ele calcula a distância euclidiana até o ponto atual e seleciona aquele que estiver mais próximo.
    - Este cliente é adicionado à rota e marcado como visitado. O processo se repete até que todos os X clientes tenham sido incluídos. Por fim, adiciona-se o trecho de retorno ao depósito. Isso gera uma “Rota Inicial” viável, mas geralmente ineficiente (com cruzamentos).
2. **Melhoria Local (Intercâmbio 2-Opt):**
    - O algoritmo tenta melhorar a rota inicial eliminando cruzamentos geométricos.
    - Ele seleciona duas arestas (trechos) da rota atual, remove-as e reconecta os quatro pontos resultantes de forma invertida (revertendo a ordem de um segmento da rota).
    - Se a nova rota resultante tiver uma distância total menor que a anterior, a mudança é aceita permanentemente.
    - O algoritmo varre todas as combinações possíveis de arestas e repete o processo iterativamente até que nenhuma troca de 2 arestas consiga reduzir a distância total (atingindo um “ótimo local”).

3. Modelagem Formal

A formulação matemática abaixo descreve o modelo exato do Problema do Caixeiro Viajante que serve de base teórica para a nossa resolução heurística.

3.1 Conjuntos e Índices

- $V = \{0\} \cup C$: Conjunto de todos os vértices (nós) do grafo do dia.
- 0: Depósito fixo (Hotel / Base de partida e chegada).
- $C = \{1, 2, \dots, X\}$: Conjunto de clientes selecionados para a visita, onde $|C| = X$.

3.2 Parâmetros

- $d_{ij} \geq 0$: Distância euclidiana entre os pontos i e j ($\forall i, j \in V, i \neq j$), calculada diretamente a partir das coordenadas (x_i, y_i) e (x_j, y_j) fornecidas no dataset. Como a malha é simétrica, $d_{ij} = d_{ji}$.

3.3 Variáveis de Decisão

$$x_{ij} \in \{0, 1\}, \qquad \forall i, j \in V, i \neq j$$

- $x_{ij} = 1$ se o deslocamento direto do nó i para o nó j faz parte da rota ótima.
- $x_{ij} = 0$ caso contrário.

3.4 Função Objetivo

O objetivo é minimizar a soma das distâncias de todos os arcos utilizados na rota fechada:

$$\text{Minimizar } Z = \sum_{i \in V} \sum_{j \in V, j \neq i} d_{ij} x_{ij}$$

3.5 Restrições

R1. Grau de saída (cada nó deve ter exatamente uma partida):

$$\sum_{j \in V, j \neq i} x_{ij} = 1, \quad \forall i \in V$$

R2. Grau de entrada (cada nó deve ter exatamente uma chegada):

$$\sum_{i \in V, i \neq j} x_{ij} = 1, \quad \forall j \in V$$

R3. Eliminação de Subciclos (Formulação MTZ - Miller-Tucker-Zemlin):

Para garantir que a solução forme um único ciclo contínuo passando pelo depósito (evitando múltiplos ciclos desconectados entre clientes), introduzimos variáveis contínuas auxiliares u_i (representando a posição sequencial de visita do nó i):

$$u_i - u_j + X \cdot x_{ij} \leq X - 1, \qquad \forall i, j \in C, i \neq j$$

$$1 \leq u_i \leq X, \qquad \forall i \in C$$

(Nota: Embora a modelagem linear inteira completa exija as restrições MTZ acima para evitar subciclos, nossa implementação computacional adota a permutação direta dos vetores de clientes nas heurísticas de Vizinho Mais Próximo e 2-Opt, o que garante estruturalmente a formação de um tour único e válido sem a necessidade de resolver o modelo matricial completo).